MLSDO Task 1: Context, Data & Model

Andrei Foitoș[S5233836][1], Andrei-George Iclodean[S6480039][1], and Yuwen Zhou[S5521351][1]

[1]Faculty of Science and Engineering, University of Groningen, The Netherlands
a.foitos@student.rug.nl
a.iclodean@student.rug.nl
y.zhou.74@student.rug.nl

December 14, 2025

1 System Scope and Context

1.1 Problem Description

Architectural Design Decisions (ADDs) represent the rationale behind the structure, behavior, and evolution of software systems. They capture trade-offs, quality attribute considerations, and the reasoning that influences architectural direction. Despite their importance, ADDs are hardly ever explicitly documented in practice. Instead, they are embedded informally in communication channels such as emails, issue trackers, and discussion threads. As a result, ADDs become implied knowledge that is difficult to trace, reuse, or validate.

Issue tracking systems, such as Jira, are widely used to coordinate software development activities. These systems contain rich information about system evolution; however, the architectural knowledge within them is difficult to identify manually due to the unstructured nature of issue descriptions, the ambiguity of terminology, and the low prevalence of explicit architectural content. The absence of systematic extraction mechanisms hampers architectural knowledge sharing, maintenance efforts, and organizational learning.

The system developed in this project aims to address this challenge by providing an automated, machine-learning–enabled platform for detecting ADDs in issue tracking systems. By supporting the classification and search of ADD-related content, the system improves accessibility, traceability, and the reuse of architectural knowledge.

1.2 Intended Users and Primary Use Cases

Intended Users

The system is designed for the following user groups:

- **Software architects**, who require access to historical architectural reasoning to support informed decision-making.
- **Software developers**, who benefit from understanding the architectural implications of tasks they implement.
- **Project managers and technical leads**, who use architectural insights to improve planning and governance.
- **Researchers**, who investigate the nature and distribution of architectural knowledge within development artifacts.

Primary Use Cases

The core use cases supported by the system are:

- **UC1: Single-Issue ADD Classification:** The user submits a single issue, and the system predicts whether it contains an ADD and, if so, its type.
- **UC2: Batch Classification:** The user submits multiple issues for asynchronous ADD detection, enabling efficient large-scale processing.
- **UC3: Keyword-Based ADD Search:** The user provides one or more keywords, and the system retrieves issues containing ADDs relevant to the queried topic.

1.3 Background: ADDs and Challenges in Issue Tracking Systems

ADDs have come to be viewed as the foundational units of software architecture, reflecting a shift from describing systems primarily through components and connectors to understanding architecture as the outcome of decision-making processes. This decision-centric view highlights that architecture is shaped less by static structures than by the reasoning, constraints, and trade-offs that guide their creation. As Jansen and Bosch (2005) and Kruchten et al. (2006) note, ADDs persist as long-lived knowledge artifacts because they capture underlying rationales that remain relevant even as implementation details evolve.

Research has proposed several classifications of ADDs, typically distinguishing among decisions about the existence and configuration of architectural elements, decisions governing component structure or organizational design, and decisions oriented toward quality attributes such as performance, security, or maintainability. These categories illustrate the breadth of concerns contained by architectural reasoning and provide a conceptual basis for studying ADDs across diverse software systems.

The representation of architectural decisions has been explored through both structured and lightweight documentation models. Formal approaches, such as the Issue-Based Information System (IBIS), emphasize argumentation and the evaluation of alternatives, while pragmatic formats like Architecture Decision Records (ADRs) aim to preserve essential information in a streamlined, developer-friendly manner. These models underscore the importance of making architectural reasoning explicit, traceable, and accessible over time.

Recent empirical studies show, however, that architectural knowledge remains widely distributed and highly dynamic in practice. Decisions often evolve incrementally and are scattered across various communication channels, complicating systematic analysis. This complexity has motivated the application of natural language processing and machine learning methods, with transformer-based models demonstrating promise in identifying ADDs within heterogeneous and unstructured development artifacts (Soliman et al., 2025). This emerging line of research lays the foundation for automated tools that support scalable architectural knowledge management.

1.4 Organizational Goals and Leading Indicators

Organizational Goals

The system supports several high-level organizational objectives:

- **OG1**: Enhance accessibility of architectural knowledge across teams.
- **OG2**: Improve traceability and consistency of architectural decisions.
- **OG3**: Reduce the time required to locate architectural reasoning in historical data.
- **OG4**: Enable data-driven analysis of architectural evolution.

Leading Indicators

Achievement of these goals can be monitored using the following indicators:

- **LI1**: Number of ADDs identified by the system per project.

- **LI2**: Reduction in manual effort required to locate architectural reasoning.
- **LI3**: Frequency of use of the ADD search interface.
- **LI4**: Improvements in model performance across successive iterations.

1.5 System Goals

The primary system objectives are as follows:

- **SG1**: Automatically classify issues with respect to ADD presence and type.
- **SG2**: Support asynchronous and scalable processing for large batches of issues.
- **SG3**: Provide efficient keyword-based retrieval of ADD-relevant issues.
- **SG4**: Ensure data quality and reproducibility through DVC and Pandera.
- **SG5**: Offer an interactive and responsive web-based interface.

1.6 User Goals

Users interacting with the system aim to:

- **UG1**: Determine rapidly whether an issue contains architectural decisions.
- **UG2**: Perform batch-level ADD detection with minimal waiting time.
- **UG3**: Search for ADDs relevant to specific architectural or technical concepts.
- **UG4**: Interpret model predictions through an intuitive interface.

1.7 Model Goals

The machine learning model developed for this project is expected to:

- **MG1**: Achieve high accuracy in classifying ADD types.
- **MG2**: Minimize false positives given the class imbalance in the dataset.
- **MG3**: Ensure reproducibility of experiments using MLflow.
- **MG4**: Generalize effectively across heterogeneous Jira ecosystems.

1.8 Key Requirements

Functional Requirements

- **FR1**: The system shall classify ADDs for individual issues.
- **FR2**: The system shall support asynchronous batch processing.
- **FR3**: The system shall support keyword-based ADD searches.
- **FR4**: The API shall expose endpoints for prediction and search.
- **FR5**: The frontend shall allow users to submit issues and view results.

Non-Functional Requirements

- **NFR1**: The system shall maintain API responsiveness during inference.
- **NFR2**: Data shall be validated through a Pandera schema.
- **NFR3**: The machine learning lifecycle shall be tracked with MLflow.
- **NFR4**: The system shall be deployable through a CI/CD pipeline.
- **NFR5**: System metrics shall be exposed through Prometheus and visualized in Grafana.

1.9 Specifications

Table 1 summarizes the technical specifications derived from the requirements.

Requirement	Specification
FR1	REST endpoint (`/predict`) returning JSON-formatted ADD predictions.
FR2	Background worker or asynchronous task manager for batch inference.
FR3	PostgreSQL full-text search or ZomboDB-based search index.
FR4	OpenAPI-compliant API documentation.
FR5	Responsive frontend with support for asynchronous updates.
NFR1	Non-ML API requests must respond within 200 ms.
NFR2	Pandera schema validates preprocessed Jira issue fields.
NFR3	MLflow logs parameters, metrics, and model artifacts.
NFR4	GitLab CI/CD runs tests, style checks, builds, and deployment.
NFR5	Prometheus metrics and Grafana dashboards for system monitoring.

Table 1: Specifications derived from system requirements

1.10 Assumptions

The system relies on the following assumptions:

- **A1**: Jira issue summaries and descriptions contain sufficient information to infer ADDs.
- **A2**: The provided dataset is representative of typical ADD occurrence patterns.
- **A3**: Users access the system through a web interface or programmatically via the API.
- **A4**: Data stored in DVC is fully preprocessed prior to model training.
- **A5**: The deployed model remains accessible to the inference service.

1.11 Fault Tree Analysis of a Key Requirement

This section presents a Fault Tree Analysis (FTA) of the key non-functional requirement **NFR1**: *The API shall remain responsive during machine learning inference.*

Top Event

API becomes unresponsive (latency exceeds acceptable threshold).

Contributing Faults

- **F1: Synchronous Inference Execution** Model inference blocks the main request-handling thread due to synchronous execution or absence of background workers.
- **F2: Resource Starvation** CPU or memory resources are exhausted during batch processing, preventing timely response generation.
- **F3: Inefficient Model or Infrastructure** Excessively large model size, absence of caching mechanisms, or insufficient container resources hinder responsive behavior.

Conclusion

The FTA highlights the need for asynchronous execution, resource isolation, and efficient model loading strategies to ensure API responsiveness during inference. These insights directly influence architectural and deployment decisions adopted in subsequent tasks.

2 Data Management and Preprocessing

The project relies on the MiningDesignDecisions (MDD) database and the JiraRepos database provided in MongoDB (Maarleveld and Dekker, 2023). The MDD database contains manually annotated Architectural Design Decision (ADD) labels for a selected subset of issues, and each annotated entry includes three binary dimensions: existence, property, and executive. With a specific "has-label" tag, the annotation was created as part of the original research study. In contrast, the JiraRepos database contains the full corpus of approximately 2.8 million raw Jira issues across several software projects. Each issue includes a summary and a description, which together form the textual input used for model training. For our task, only issues appearing in the MDD annotation set serve as labeled data.

To ensure reproducibility and proper versioning, all processed datasets were managed using Data Version Control (DVC). After extracting and merging labeled issues with their corresponding Jira entries, the resulting dataset was saved into a CSV file and tracked through DVC. The preprocessing step includes data cleaning, filtering, and transformation steps to prepare the textual fields for machine learning models. Jira descriptions often contain HTML markup, formatting tags, and large code or log fragments. To reduce noise, we removed the HTML elements and Jira-specific formatting, and irregular whitespace was normalized. Summary and description fields were converted to plain text, and issues lacking meaningful textual content were filtered out. During data integration, only about 6,225 issues with valid boolean ADD labels were retained. Exploratory analysis of this retained subset revealed a significant class imbalance, with non-ADD issues outnumbering ADD-positive issues. This distributional imbalance highlighted the necessity of prioritizing metrics like the F1-score and Recall rather than simple Accuracy for model evaluation.

We also address potential mistakes and biases inherent in the data source. Since the dataset is derived exclusively from open-source Apache projects, it likely over-represents the development culture, terminology, and English-language documentation specific to that community. Consequently, the model may underperform on proprietary software or projects with different documentation standards.

To guarantee data quality and implement a "design for mistakes" strategy, a schema-based validation step was implemented using the Pandera library (Bantilan, 2020). Pandera enforces constraints on column types, requiring that project identifiers and issue IDs are valid strings, ADD labels are boolean, and textual fields are properly defined. A global integrity rule ensures that at least one of summary or description is non-empty. This validation framework serves as a robust error-handling mechanism: rather than crashing the pipeline upon encountering malformed data, the system automatically identifies and filters out non-conforming records, ensuring that the training stage proceeds only with valid, high-quality samples.

3 Model Development and Performance Tracking

The classification of Architectural Design Decisions (ADDs) from unstructured software issue trackers presents a significant Natural Language Processing (NLP) challenge. To address this, we employed a deep learning approach leveraging the Transformer architecture.

Initially, we prototyped the system using TinyBERT (`google/bert_uncased_L-2_H-128_A-2`) [1] (Turc et al., 2019) to validate the end-to-end reproducibility of the pipeline with minimal computational overhead. However, preliminary evaluations indicated that this architecture lacked the capacity to capture the complex semantic nuances of architectural discussion. The prototype achieved a modest Recall of 0.68 and an F1-Score of 0.69, suggesting it missed approximately 32% of relevant design decisions. Consequently, we transitioned to DistilBERT (`distilbert-base-uncased`) [2] (Sanh et al., 2019) as the classification model. This selection was justified by DistilBERT's ability to retain approximately 97% of the performance of the full BERT-Base model while reducing the parameter count by 40%. This upgrade proved effective: the transition resulted in a 12.5% absolute improvement in Recall (rising to 0.80) and a 6.8% increase in F1-Score (0.76). While the inference latency increased, DistilBERT processes approximately 50 samples per second compared to TinyBERT's 890, the trade-off was deemed necessary to achieve a reliable detection rate for architectural mining tasks. The training process was implemented using the Hugging Face Transformers library (Wolf et al., 2020),

[1] https://huggingface.co/google/bert_uncased_L-2_H-128_A-2
[2] https://huggingface.co/distilbert/distilbert-base-uncased

fine-tuning the pre-trained DistilBERT model on the labeled dataset extracted from the JiraRepos and MiningDesignDecisions databases.

The problem was formulated as a binary classification task, where the model predicts the probability of an issue belonging to the "ADD" class based on the concatenation of its summary and description fields. The dataset, consisting of 6,225 validated records, was tokenized with a maximum sequence length of 512 tokens to capture long-form technical descriptions. We employed a 70-15-15 split for training, validation, and testing, respectively. The model was optimized using the AdamW optimizer (**?**) with a learning rate of 2×10^{-5}. Due to the increased memory usage of DistilBERT, the batch size was adjusted to 8, and training was extended to 4 epochs to ensure convergence. To ensure rigorous experiment tracking and lifecycle management, MLflow (Zaharia et al., 2018) was fully integrated into the training workflow. This integration enabled the automatic logging of essential hyperparameters, training configurations, and evaluation metrics for every experimental run. Furthermore, the trained model artifacts, including the tokenizer and model weights, were registered within the MLflow Model Registry under the identifier add-classifier This versioning strategy establishes a clear history for the model, allowing stakeholders to trace specific model versions back to the exact code and data version (via DVC) used to produce them, effectively satisfying the requirement for reproducible model lifecycle management. Initial performance evaluation on the held-out test set achieved an Accuracy of 0.79 and an F1-Score of 0.76. The model achieved a Precision of 0.72 and a Recall of 0.80. These metrics demonstrate a balanced classifier that is significantly more robust than the initial prototype. The high recall is particularly valuable for this domain, as it minimizes the risk of overlooking critical architectural decisions during automated analysis. A comparison between the general characteristics of the two versions of ADD classifier and their performance metrics can be seen in Table 2 and Table 3, respectively. Model key characteristics, uses, bias, risks, and limitations can also be seen in the MODELCARD.

Attribute	**Baseline (Prototype)**	**Selected Model (Current)**
Model Identifier	`google/bert_uncased_L-2_H-128_A-2`	`distilbert-base-uncased`
Architecture	TinyBERT (2 Layers, 128 Hidden)	DistilBERT (6 Layers, 768 Hidden)
Parameters	$\approx$ 4.4 Million	$\approx$ 66 Million
Inference Speed	$\approx$ 890 samples/sec	$\approx$ 50 samples/sec
MLflow Version	`add-classifier` (v2)	`add-classifier` (v3)

Table 2: Comparison of Candidate Model Architectures

Metric	**TinyBERT**	**DistilBERT**	**Improvement**
Accuracy	0.74	**0.79**	+5%
F1-Score	0.69	**0.76**	+7%
Precision	0.71	**0.72**	+1%
Recall	0.68	**0.80**	+12%

Table 3: Performance Comparison: Baseline vs. Selected Model

Intended Use & Limitations The model is intended to assist researchers and practitioners in automatically mining architectural knowledge from large-scale issue trackers. It is fine-tuned on open-source projects (e.g., Apache) and may not generalize perfectly to proprietary software with different terminologies or documentation styles. While the recall has improved to 0.80, the model still generates false positives (Precision 0.72), implying that human verification remains recommended for precision-critical audits.

Ethical Considerations The training data is derived exclusively from specific open-source communities, which may introduce bias towards English-language issues and specific development cultures. There is a risk that the model may under-represent design decisions made in non-standard formats or by non-native English speakers.

References

Bantilan, N. (2020). pandera: Statistical data validation of pandas dataframes. In *Proceedings of the 19th Python in Science Conference*, pages 116–124.

Jansen, A. and Bosch, J. (2005). Software architecture as a set of architectural design decisions. In *Proceedings of the 5th Working IEEE/IFIP Conference on Software Architecture (WICSA'05)*, pages 109–120. IEEE.

Kruchten, P., Lago, P., and van Vliet, H. (2006). Building up and reasoning about architectural knowledge. In *Quality of Software Architectures (QoSA)*, volume 4214 of *Lecture Notes in Computer Science*, pages 43–58. Springer.

Maarleveld, J. and Dekker, A. (2023). Developing deep learning approaches to find and classify architectural design decisions in issue tracking systems. Master's thesis, University of Groningen.

Sanh, V., Debut, L., Chaumond, J., and Wolf, T. (2019). Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.

Soliman, M., Albonico, M., Malavolta, I., and Wortmann, A. (2025). Mining software repositories for software architecture — a systematic mapping study. *Information and Software Technology*, 181:107677.

Turc, I., Chang, M.-W., Lee, K., and Toutanova, K. (2019). Well-read students learn better: On the importance of pre-training compact models. *arXiv preprint arXiv:1908.08962*.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., et al. (2020). Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45.

Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nykazim, T., Ogilvie, P., Parkhe, M., et al. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4):39–45.